Reviewer Pack — Grothendieck-Teichmüller Group Action Count and MCSP Depth

Entry a7ecb4638b34 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 14:07:45 UTC

Grothendieck-Teichmüller Group Action Count and MCSP Depth

Entry ID: `a7ecb4638b34` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 14:07:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Grothendieck-Teichmüller Groups **Field B** (complexity object): Meta-complexity: MCSP (Meta-Complexity)

Statement:

> {'statement_1': 'The action count of the Grothendieck-Teichmüller group on a given finite set of algebraic structures is asymptotically equivalent to the depth of the Meta-Complexity of that structure.', 'statement_2': 'Formally, for a structure S with MCSP depth D(S), there exists a constant C such that the action count of the Grothendieck-Teichmüller group on S is bounded by C * D(S).', 'statement_3': 'This equivalence implies that problems expressible in Meta-complexity have an associated Grothendieck-Teichmüller group action that can be used to bound their complexity.'}

Rationale (proposer's reasoning):

> {'rationale_1': 'The Grothendieck-Teichmüller groups encode the symmetries of algebraic structures, and their actions are known to exhibit rich combinatorial properties.', 'rationale_2': 'By linking these group actions to Meta-complexity, we aim to provide new insights into the computational complexity of problems expressible in this formalism.', 'rationale_3': 'This bridge could potentially reveal deep connections between algebraic geometry and complexity theory.'}

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d3b1d6d1533266d9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the ratio of action count to MCSP depth for all structures is bounded by a constant C with a mean ratio ≤ 1.5 across 30 random seeds, and falsified if any seed produces a ratio > 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Grothendieck-Teichmüller Groups' AND 'MCSP depth' - 'action count Grothendieck-Teichmüller Group' AND 'Meta-complexity' - 'algebraic geometry Grothendieck-Teichmüller Groups' AND 'problem complexity Meta-complexity'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_structure(n):
        # Placeholder for structure generation logic
        return [i for i in range(1, n+1)]

    def compute_action_count(structure):
        # Placeholder for action count computation logic
        return sum(structure)

    def compute_mcsp_depth(structure):
        # Placeholder for MCSP depth computation logic
        return len(structure)

    structures = [generate_structure(n) for n in [5, 10, 15, 20, 30, 40]]
    results = []

    for structure in structures:
        action_count = compute_action_count(structure)
        mcsp_depth = compute_mcsp_depth(structure)

        if mcsp_depth == 0:
            continue

        ratio = Fraction(action_count, mcsp_depth)
```

```
        results.append({
            "structure": structure,
            "action_count": action_count,
            "mcsp_depth": mcsp_depth,
            "ratio": ratio
        })

    if not results:
        return {
            "metric_name": "action_count_to_mcsp_ratio",
            "metric_value": 0.0,
            "instances_tested": len(structures),
            "conjecture_holds": False,
            "counterexample": "No valid structures generated"
        }

    max_ratio = max(r["ratio"] for r in results)
    conjecture_holds = max_ratio <= Fraction(15, 10)

    return {
        "metric_name": "action_count_to_mcsp_ratio",
        "metric_value": sum(r["ratio"] for r in results) / len(results),
        "instances_tested": len(structures),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"max_ratio={max_ratio}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"max_ratio={results[0]['counterexample']}\" firs
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
6, 'conjecture_holds': False, 'counterexample': 'max_ratio=41/2'}
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
```

```
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes
TRIAL: {'metric_name': 'action_count_to_mcsp_ratio', 'metric_value': Fraction(21, 2), 'instances_tes

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac549fd5.py", line 84, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac549fd5.py", line 84, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that for all tested instances, the ratio of action count to MCSP depth exceeds the threshold of 2, which contradicts the con | next: Investigate the counterexamples in detail to understand why the ratios are so high and whether there is a pattern or specific type of structure causing this discrepancy.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15968
2	preregistration	ollama_remote	glm4:latest	0	0	9654
3	novelty	ollama_remote	glm4:latest	0	0	9099
4	novelty	ollama_remote	glm4:latest	0	0	12334
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16459
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6879
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8284
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8317
9	judge	ollama_remote	glm4:latest	0	0	11791

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 98785 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a7ecb4638b34.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a7ecb4638b34.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a7ecb4638b34.tar.gz` (if generated)